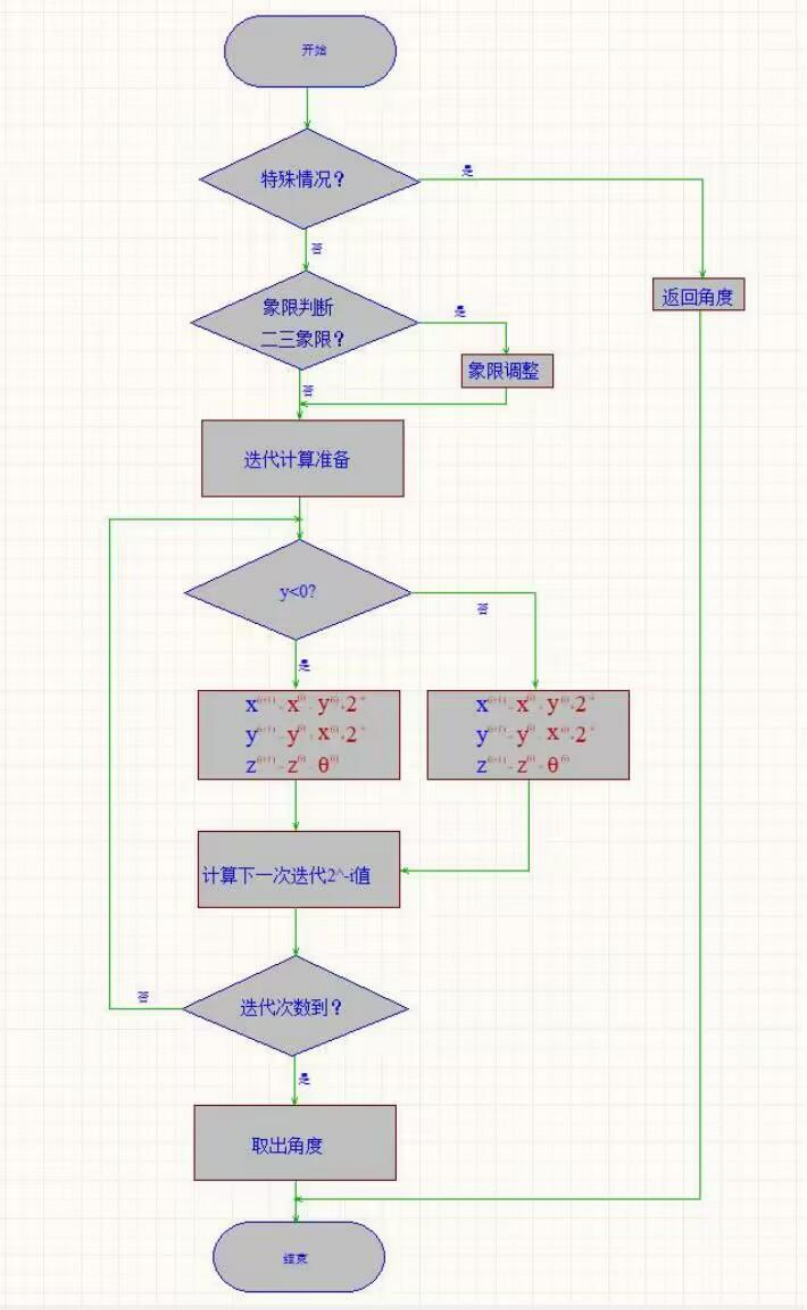
62-FOC 15

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。

上次课呢，我们把求平方根的函数分析完了。接着我们来分析下 Cordic 数字迭代算法的代码实现。

```
extern unsigned int CalSqrt(unsigned int x);
extern signed int CalAtan(signed int y,signed int x);
/*=============================================================
```

大家看下，这个函数呢，同样是使用汇编来编写的。函数带有两个形参及一个返回值返回值就是我们要求解的角度。x 跟 y 的话，如果求超前角的话，就是 Ud 跟 Uq。为了很好的分析代码，我们再给一个流程图。



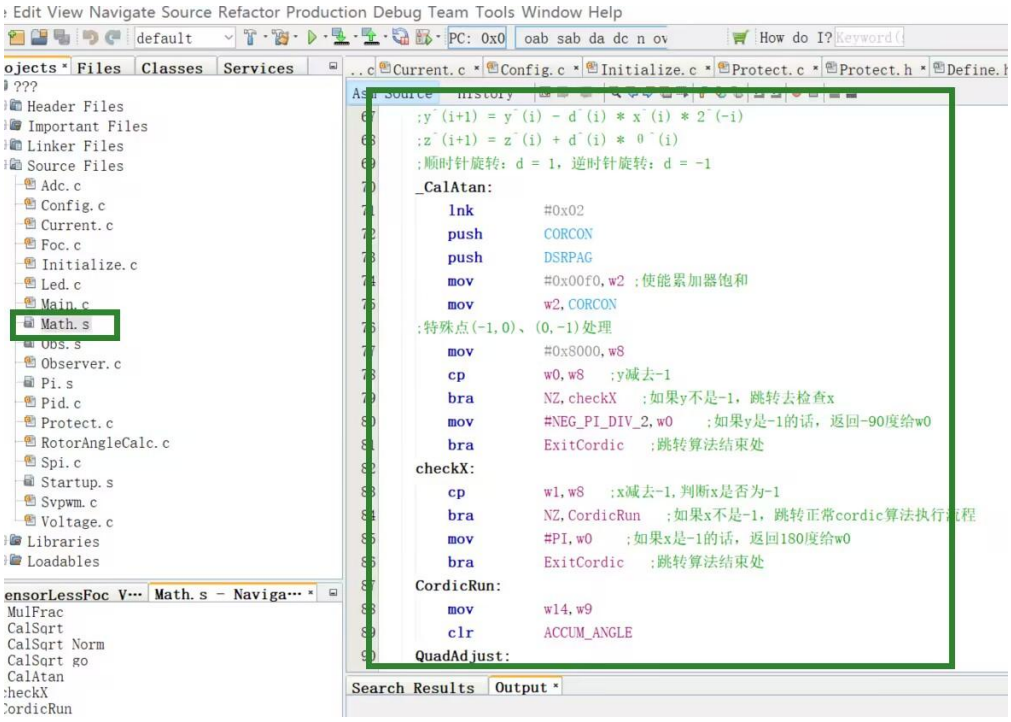
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

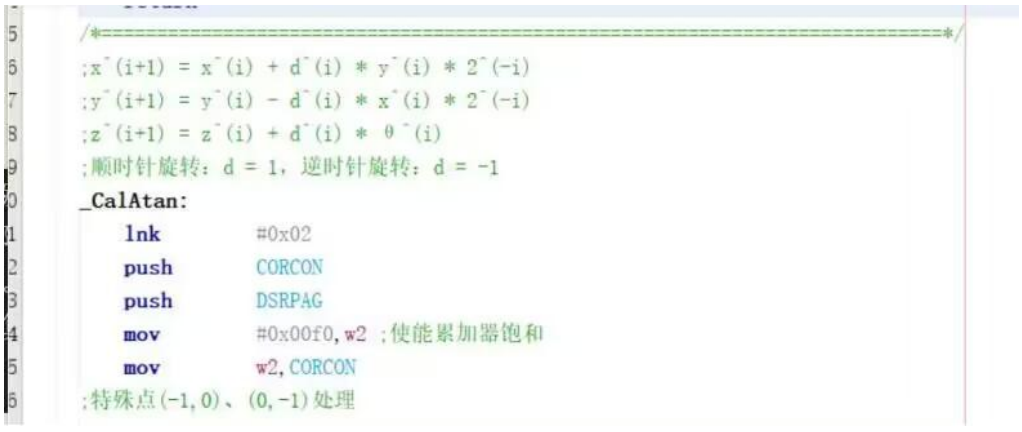
大家其实可以看到，函数进来，先处理一下特殊点的情况，如果是特殊点，则会返回对应的角度，函数就结束了。

如果不是特殊点的话，接着再进行象限判断，如果是第二象限或者第三象限的话，就需要进行象限调整。象限调整完之后，就要开始计算迭代方程，并进行角度累加。当迭代次数到了设定值之后，就可以取出角度累加器中的角度。这就是我们之前分析的 Cordic 数字迭代算法对应的流程图。下面呢，我们就根据这个流程图来分析下对应的代码。

```
.global  _MulFrac
.global  _CalSqrt
.global  _CalAtan
/*=====================================================
```

同样的，这个函数要在 C 文件中进行调用，所以需要进行全局声明。





本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

大家看下，旋转中使用到的迭代方程，我们用注释的形式写出来了，这样方便后面使用。

```
_CalAtan:
    lnk         #0x02
    push        CORCON
    push        DSRPAG
    mov         #0x00f0,w2  ;使能累加器饱和
    mov         w2,CORCON
;特殊点(-1,0)、(0,-1)处理
    mov         #0x8000,w8
    cp          w0,w8       ;y减去-1
    bra         NZ,checkX   ;如果y不是-1，跳转去检查x
    mov         #NEG_PI_DIV_2,w0    ;如果y是-1的话，返回-90度给w0
    bra         ExitCordic  ;跳转算法结束处
checkX:
```

这里，我们接触了新的指令了。

LNK	#lit14	分配堆栈帧	1	1	267

这里说指令的意思是分配堆栈帧。

其实，通俗理解，就是我们在堆栈空间中，申请地址空间。大家也可以理解为 C 语言函数中的局部变量，因为，变量本身就是一段地址空间。只不过，我们这里临时在堆栈空间中申请一个地址，来临时存放数据。

问：lnk　#0x02 中的#0x02 是什么意思？

答：在堆栈空间中申请两个地址长度的临时空间。

当这个函数执行结束之后，我们又把这个地址给释放掉。这就是 lnk 指令的作用。

ULNK	释放堆栈帧	1	1	435

释放的话，就是 ULNK 指令。这两条指令是搭配使用的，一个 LNK 必须有一个 ULNK。

```
ExitCordic:
    pop         DSRPAG
    pop         CORCON
    ulnk
    return
/*=====================================================
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

大家看下，这是函数执行结束之后　加上 ULNK 指令，来释放临时申请的地址空间。大家就把这个临时空间当成一个局部变量来理解就行。

```
_CalAtan:
    lnk     #0x02
    push    CORCON
    push    DSRPAG
    mov     #0x00f0,w2 ;使能累加器饱和
    mov     w2,CORCON
;特殊点(-1,0)、(0,-1)处理
    mov     #0x8000,w8
    cp      w0,w8   ;y减去-1
    bra     NZ,checkX   ;如果y不是-1，跳转去检查x
    mov     #NEG_PI_DIV_2,w0    ;如果y是-1的话，返回-90度给w0
    bra     ExitCordic  ;跳转算法结束处
checkX:
```

因为我们后面会查表，所以会使用到 DSRPAG 寄存器。所以，函数开始先把 DSRPAG 寄存器的原始值压入堆栈保存。同样，我们会修改 CORCON 寄存器的内容，以此来修改累加器的饱和功能。所以，也需要把 CORCON 寄存器压入堆栈进行保存。因为我们后面的迭代方程的计算，会使用到累加器。而因为迭代计算，累加器中存储的数据就不断在累加。当然，正常情况下，不会让累加器累加溢出，为了防止异常情况出现，我们可以把累加器设定好饱和功能。
也就是说，当累加器累加到最大值，再累加的话，就会限制为累加器的最大值。

bit 7	**SATA**：ACCA 饱和使能位 (1) 1 = 使能累加器 A 饱和 0 = 禁止累加器 A 饱和
bit 6	**SATB**：ACCB 饱和使能位 (1) 1 = 使能累加器 B 饱和 0 = 禁止累加器 B 饱和
bit 5	**SATDW**：DSP 引擎的数据空间写饱和使能位 (1) 1 = 使能数据空间写饱和 0 = 禁止数据空间写饱和
bit 4	**ACCSAT**：累加器饱和模式选择位 (1) 1 = 9.31 饱和 （超饱和） 0 = 1.31 饱和 （正常饱和）

大家看下，CORCON 寄存器的 bit4~bit7 都是跟饱和相关的。我们把这 4 位都设置为 1，就把饱和打开了。和数据格式有关内容，可以参考下面内容。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

4.11 DSP 数据格式（dsPIC30F、dsPIC33F 和 dsPIC33E 器件）

4.11.1 整数和小数数据

dsPIC30F、dsPIC33F 和 dsPIC33E 器件支持整数和小数数据类型。整型数据固有表示形式为有符号的二进制补码值，其中最高位定义为符号位。通常来说，N 位二进制补码整数的范围为 -2^{N-1} 至 $2^{N-1}-1$。对于 16 位整数，数据范围为 -32768（0x8000）至 32767（0x7FFF），包括 0。对于 32 位整数，数据范围为 -2,147,483,648（0x8000 0000）至 2,147,483,647（0x7FFF FFFF）。

小数数据表示为二进制补码数，其中最高位定义为符号位，小数点隐含于符号位之后。这种格式通常被称为 1.15（或 Q15）格式，其中 1 是用来表示数据的整数部分的位数，而 15 是用来表示小数部分的位数。采用这种隐含小数点的 N 位二进制补码表示的小数范围为 -1.0 至 $(1-2^{1-N})$。对于 16 位小数，1.15 数据格式的范围为 -1.0（0x8000）至 0.999969482（0x7FFF），包括 0.0 且精度为 3.05176×10^{-5}。在普通饱和模式下，32 位累加器使用 1.31 数据格式，可将精度提高至 4.6566×10^{-10}。

通过使用累加器最高字节寄存器（ACCxU）的 8 个位作为警戒位扩展了累加器的动态范围。如果存放在累加器中的值发生溢出超出了 32 位的限制，此时将借助警戒位实现 DSP 算法。当 ACCSAT 位（CORCON<4>）置 1 时，将使能该模式并将累加器扩展为 40 位。当禁止累加器饱和时也可使用警戒位。此时，累加器可支持的整数范围为 -5.498×10^{11}（0x80 0000 0000）至 5.498×10^{11}（0x7F FFFF FFFF）。在小数模式下，累加器的警戒位不会改变小数点的位置且 40 位累加器使用 9.31 小数格式。注意，所有小数运算的结果都存入 40 位累加器，并采用 1.31 小数点格式进行对齐。在整数模式下，警戒位仅增加了累加器的动态范围。9.31 小数格式的范围为 -256.0（0x80 0000 0000）至（$256.0-4.65661\times10^{-10}$）（0x7F FFFF FFFF）。表 4-10 指明了对于 16 位、32 位和 40 位寄存器，dsPIC30F/33F/33E 器件中整数和小数的范围和精度。

应注意，除 DSP 乘法操作之外，ALU 对于整数和小数数据的操作是相同的。也就是说，两个整数相加将产生与两个小数相加相同的结果（二进制数）。唯一的差别在于用户如何对该结果进行解释。然而，DSP 操作执行的乘法是不同的。在这些指令中，数据格式的选择是通过设定 IF 位（CORCON<0>）来进行的。因此必须对其进行设定，将其设定为 0 选择小数模式，设定为 1 选择整数模式。因为 dsPIC30F/33F/33E 小数使用隐含的小数点，因此需要进行上述设定。在整数模式下，两个 16 位整数相乘将产生一个 32 位整数结果。然而，两个 1.15 格式的值相乘将产生一个 2.30 格式的结果。由于 dsPIC30F、dsPIC33F 和 dsPIC33E 器件的累加器采用 1.31 数据格式，小数模式的 DSP 乘法执行过程也包括一个左移一位的操作以保持小数点能够正确对齐。该特点将使 DSP 乘法器的精度降低至 2^{-30}，但对于计算将不会产生其他影响（例如，0.5 x 0.5 = 0.25）。

表 4-10： dsPIC30F/33F/33E 的数据范围

寄存器宽度	整数范围	小数范围	小数精度
16 位	-32768 至 32767	-1.0 至 $(1.0-2^{-15})$	3.052×10^{-5}
32 位	-2,147,483,648 至 2,147,483,647	-1.0 至 $(1.0-2^{-31})$	4.657×10^{-10}
40 位	-549,755,813,888 至 549,755,813,887	-256.0至$(256.0-2^{-31})$	4.657×10^{-10}

```
_CalAtan:
    lnk         #0x02
    push        CORCON
    push        DSRPAG
    mov         #0x00f0,w2  ;使能累加器饱和
    mov         w2,CORCON
;特殊点(-1,0)、(0,-1)处理
    mov         #0x8000,w8
```

大家看下这两句话，就是实现把 CORCON 寄存器的 bit4~bit7 设置为 1，这样的话，前期的准备就做好了。

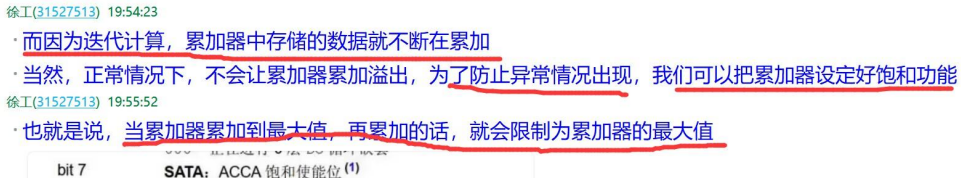
下面开始判断特殊点了。

```
_CalAtan:
    lnk         #0x02
    push        CORCON
    push        DSRPAG
    mov         #0x00f0,w2  ;使能累加器饱和
    mov         w2,CORCON
;特殊点(-1,0)、(0,-1)处理
    mov         #0x8000,w8
    cp          w0,w8       ;y减去-1
    bra         NZ,checkX   ;如果y不是-1，跳转去检查x
    mov         #NEG_PI_DIV_2,w0    ;如果y是-1的话，返回-90度给w0
    bra         ExitCordic  ;跳转算法结束处
checkX:
```

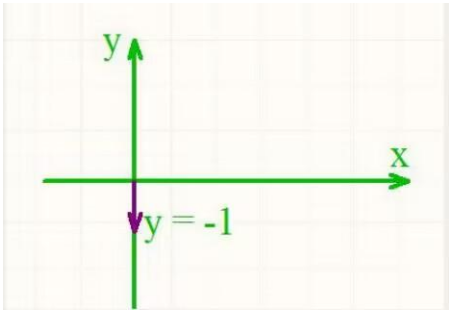
首先呢，我们先来判断 y 是不是为-1？

问：使能累加器 加个饱和是什么意思？

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



答：简单地说，就是加到最大值就不再加了。



如果 y 是-1 的话，直接给一个-90° 程序就结束。那么，-1 的话，转换为 Q15 的格式，就对应-32768。



那么，16 位的 0x8000 就对应的是-32768。所以，我们判断 y 是不是-1，就是用 y 跟-32768 比较。



如果判断出来 y 是-1，则会执行蓝框的代码，直接返回-90° ，算法结束。如果 y 不是-1，则再判断另一个特殊点，程序就会跳转到蓝色箭头对应的标号。

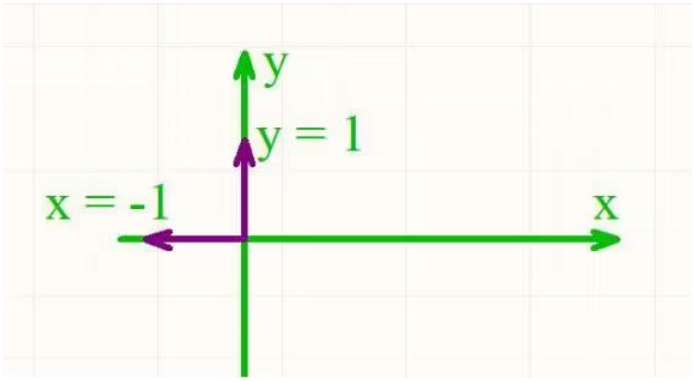
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*=============================================
;符号定义
    .equ        PI, 0x7FFF
    .equ        PI_DIV_2, 0x3FFF
    .equ        NEG_PI_DIV_2, 0xC000
    .equ        ACCUM_ANGLE, w7
```

-90° 的话，转换为 Q15 就是 0xC000.　这里只不过用符号来代替了。

```
_CalAtan:
    lnk         #0x02
    push        CORCON
    push        DSRPAG
    mov         #0x00f0, w2 ;使能累加器饱和
    mov         w2, CORCON
;特殊点(-1, 0)、(0, -1)处理
    mov         #0x8000, w8
    cp          w0, w8      ;y减去-1
    bra         NZ, checkX    ;如果y不是-1，跳转去检查x
    mov         #NEG_PI_DIV_2, w0     ;如果y是-1的话，返回-90度给w0
    bra         ExitCordic    ;跳转算法结束处
checkX:
    cp          w1, w8      ;x减去-1, 判断x是否为-1
    bra         NZ, CordicRun     ;如果x不是-1，跳转正常cordic算法执行流程
    mov         #PI, w0     ;如果x是-1的话，返回180度给w0
    bra         ExitCordic    ;跳转算法结束处
CordicRun:
    mov         w14, w9
```

接着呢，就来判断 x 特殊点。



如果判断出 x 是-1，则会直接返回 180°

```
    bra         ExitCordic    ;跳转算法结束处
checkX:
    cp          w1, w8      ;x减去-1, 判断x是否为-1
    bra         NZ, CordicRun     ;如果x不是-1，跳转正常cordic算法执行流程
    mov         #PI, w0     ;如果x是-1的话，返回180度给w0
    bra         ExitCordic    ;跳转算法结束处
CordicRun:
```

同样的，如果是-1 的话，就执行红框的代码，如果不是-1，说明不是特殊点，就会跳转到红色箭头。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
;顺时针旋转：d = 1，逆时针旋转：d = -1
_CalAtan:
    lnk         #0x02
    push        CORCON
    push        DSRPAG
    mov         #0x00f0,w2 ;使能累加器饱和
    mov         w2,CORCON
;特殊点(-1,0)、(0,-1)处理
    mov         #0x8000,w8
    cp          w0,w8    ;y减去-1
    bra         NZ,checkX    ;如果y不是-1，跳转去检查x
    mov         #NEG_PI_DIV_2,w0     ;如果y是-1的话，返回-90度给w0
    bra         ExitCordic    ;跳转算法结束处
checkX:
    cp          w1,w8    ;x减去-1,判断x是否为-1
    bra         NZ,CordicRun    ;如果x不是-1，跳转正常cordic算法执行流程
    mov         #PI,w0     ;如果x是-1的话，返回180度给w0
    bra         ExitCordic    ;跳转算法结束处
CordicRun:
```

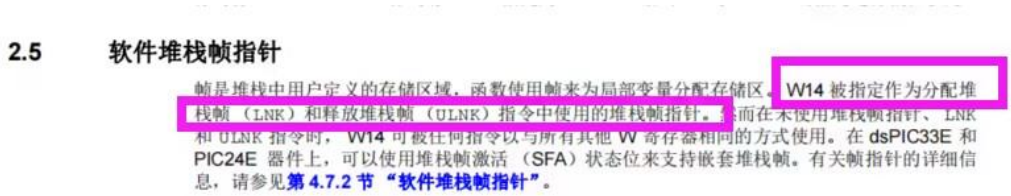
所以，这段代码就是实现了一些现场保护，累加器饱和设置以及特殊点判断处理功能。思路肯定要理解，思路理解了，大家可以用 C 实现看看。能够用 C 实现出来，就说明大家把这个迭代的思路理解清楚了。

我们继续往下讲了哈。

```
CordicRun:
    mov         w14,w9
    clr         ACCUM_ANGLE
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整 第二象限调整到第一象限，第三象限调整到第四象限
    cp0         w1
    bra         NN,CordicSetup
    mov         w1,[w9]
    cp0         w0
    bra         LE,QuadIIIQuadAdjust
    mov         w0,w1
    neg         [w9],w0
    mov         #PI_DIV_2,ACCUM_ANGLE  ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra         CordicSetup
QuadIIIQuadAdjust:
    neg         w0,w1
    mov         [w9],w0
    mov         #NEG_PI_DIV_2,ACCUM_ANGLE ;第三象限变到第四象限，逆时针旋转90度，此时初始角度是-90度
CordicSetup:
    lac         w0,#1,B
```

大家看下啊，我们申请的临时空间对应的地址是存在 w14 寄存器中的。为什么是这样呢？可以看下面红框内容。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



2.5 软件堆栈帧指针

帧是堆栈中用户定义的存储区域，函数使用帧来为局部变量分配存储区。W14 被指定作为分配堆栈帧（LNK）和释放堆栈帧（ULNK）指令中使用的堆栈帧指针。因而在未使用堆栈帧指针、LNK 和 ULNK 指令时，W14 可被任何指令以与所有其他 W 寄存器相同的方式使用。在 dsPIC33E 和 PIC24E 器件上，可以使用堆栈帧激活（SFA）状态位来支持嵌套堆栈帧。有关帧指针的详细信息，请参见**第 4.7.2 节“软件堆栈帧指针”**。

这里呢，把 w14 的内容传送给 w9，后面我们可以访问 w9 存储地址中的数据。至于为什么用 w9？这个跟 DSP 指令有关，讲到对应的部分，我们再来看啊。这里，大家先理解它是一个地址数据的赋值就行。接着呢，使用 clr 指令来清 0 角度累加器。

```
/*==========================================================
;符号定义
    .equ        PI, 0x7FFF
    .equ        PI_DIV_2, 0x3FFF
    .equ        NEG_PI_DIV_2, 0xC000
    .equ        ACCUM_ANGLE, w7
```

这里的角度累加器呢，是用 w7 的， 改一个名字呢，其实是为了看上去直观一些。

```
CordicRun:
    mov         w14, w9
    clr         ACCUM_ANGLE
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整
    cp0         w1
```

这里，清 0 角度累加器是为了后面计算角度迭代方程做准备的。接下来就是象限判断了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
CordicRun:
    mov         w14,w9
    clr         ACCUM_ANGLE
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整 第二象限调整到第一象限，第三象限调整到第四象限
    cp0         w1
    bra         NN,CordicSetup
    mov         w1,[w9]
    cp0         w0
    bra         LE,QuadIIIQuadAdjust
    mov         w0,w1
    neg         [w9],w0
    mov         #PI_DIV_2,ACCUM_ANGLE   ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra         CordicSetup
QuadIIIQuadAdjust:
    neg         w0,w1
    mov         [w9],w0
```

首先呢，需要先判断矢量在第二象限还是在第三象限， 这里呢，我们可以判断 x 的正负，就知道是不是在二三象限了。

```
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整 第
    cp0         w1
    bra         NN,CordicSetup
    mov         w1,[w9]
    cp0         w0
    bra         LE,QuadIIIQuadAdjust
    mov         w0,w1
    neg         [w9],w0
    mov         #PI_DIV_2,ACCUM_ANGLE   ;第二象限变到第一象限
    bra         CordicSetup
QuadIIIQuadAdjust:
    neg         w0,w1
```

w1 存放的就是形参 x 的值，这里用 x 跟 0 来比较，如果小于 0，就说明在二三象限，如果大于 0 就在一四象限。

```
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整 第二象限调整到第一象限，第三象限调整到第四象限
    cp0         w1
    bra         NN,CordicSetup
    mov         w1,[w9]
    cp0         w0
    bra         LE,QuadIIIQuadAdjust
    mov         w0,w1
    neg         [w9],w0
    mov         #PI_DIV_2,ACCUM_ANGLE   ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra         CordicSetup
QuadIIIQuadAdjust:
    neg         w0,w1
    mov         [w9],w0
    mov         #NEG_PI_DIV_2,ACCUM_ANGLE ;第三象限变到第四象限，逆时针旋转90度，此时初始角度是-90度
CordicSetup:
    lac         w0,#1,B
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

大家看，N 标志位是非负标志。N 标志位是 1 的话，就说明是负数，N 标志位是 0 的话，就说明是非负数。NN 的话，也就是 N 标志位不是 1，也就是结果是非负数就会跳转到红色的箭头，此时呢，也就说明矢量不在二三象限。

```
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整 第二象限调整到第一象限，第三象限调整到第四象限
    cp0         w1
    bra         NN,CordicSetup
    mov         w1,[w9]
    cp0         w0
    bra         LE,QuadIIIQuadAdjust
    mov         w0,w1
    neg         [w9],w0
    mov         #PI_DIV_2,ACCUM_ANGLE   ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra         CordicSetup
QuadIIIQuadAdjust:
    neg         w0,w1
    mov         [w9],w0
    mov         #NEG_PI_DIV_2,ACCUM_ANGLE ;第三象限变到第四象限，逆时针旋转90度，此时初始角度是-90度
CordicSetup:
```

如果说不跳转，而是执行红框的部分，也就说明矢量在二三象限。接着呢，还要再来判断具体是在第二象限还是第三象限？那么，既然已经知道是在二三象限了，就会涉及到旋转 90°，旋转 90° 的话，就是坐标的调整。

```
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整 第二象限调整到第一象限，第三象限调整到第四象限
    cp0         w1
    bra         NN,CordicSetup
    mov         w1,[w9]
    cp0         w0
    bra         LE,QuadIIIQuadAdjust
    mov         w0,w1
    neg         [w9],w0
    mov         #PI_DIV_2,ACCUM_ANGLE   ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra         CordicSetup
QuadIIIQuadAdjust:
```

所以啊，这里我们申请的地址空间就派上用场了，先把 x 临时存在这个地址空间中。因为坐标调整的话，需要有第三个存储空间，用于存储交换值。

```
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整 第二象限调整到第一象限，第三象限调整到第四象限
    cp0         w1
    bra         NN,CordicSetup
    mov         w1,[w9]
    cp0         w0
    bra         LE,QuadIIIQuadAdjust
    mov         w0,w1
    neg         [w9],w0
    mov         #PI_DIV_2,ACCUM_ANGLE   ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra         CordicSetup
QuadIIIQuadAdjust:
    neg         w0,w1
    mov         [w9],w0
    mov         #NEG_PI_DIV_2,ACCUM_ANGLE ;第三象限变到第四象限，逆时针旋转90度，此时初始角度是-90度
CordicSetup:
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

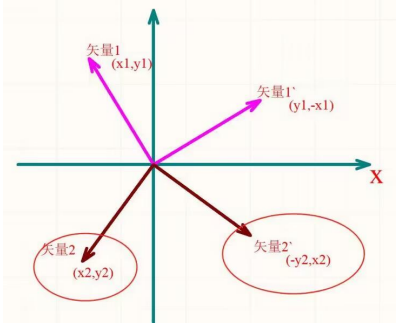
接着呢，我们来判断 y 的正负，就可以知道矢量是在第二象限还是第三象限了。

BRA	LE,Expr	如果小于或等于则转移	1	1 (2)(1,8)	137

这里使用的是一条小于或等于转移指令，如果 y 小于 0，则说明矢量在第三象限。

```
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整 第二象限调整到第一象限，第三象限调整到第四象限
    cp0         w1
    bra         NN,CordicSetup
    mov         w1,[w9]
    cp0         w0
    bra         LE,QuadIIIQuadAdjust
    mov         w0,w1
    neg         [w9],w0
    mov         #PI_DIV_2,ACCUM_ANGLE  ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra         CordicSetup
QuadIIIQuadAdjust:
    neg         w0,w1
    mov         [w9],w0
    mov         #NEG_PI_DIV_2,ACCUM_ANGLE ;第三象限变到第四象限，逆时针旋转90度，此时初始角度是-90度
CordicSetup:
    lac         w0,#1,B
```

第三象限的话，坐标调整就从红色箭头的地方开始执行。



具体调整的思路就是上图这样。也就是原始坐标的 y 取相反数之后，作为新坐标的 x，原始的 x 作为新坐标的 y，这样就实现了矢量逆时针旋转 90°，到了第四象限。

```
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整 第二象限调整到第一象限，第三象限调整到第四象限
    cp0         w1
    bra         NN,CordicSetup
    mov         w1,[w9]
    cp0         w0
    bra         LE,QuadIIIQuadAdjust
    mov         w0,w1
    neg         [w9],w0
    mov         #PI_DIV_2,ACCUM_ANGLE  ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra         CordicSetup
QuadIIIQuadAdjust:
    neg         w0,w1
    mov         [w9],w0
    mov         #NEG_PI_DIV_2,ACCUM_ANGLE ;第三象限变到第四象限，逆时针旋转90度，此时初始角度是-90度
CordicSetup:
    lac         w0,#1,B
```

neg 的话，就是取 w0 的相反数。然后存到 w1 中，w9 寄存器中存储了临时变量的地址，所以取地址中的数据，就是原始的 x，把这个值送到 w0 中，这样也就实现了坐标的交换。

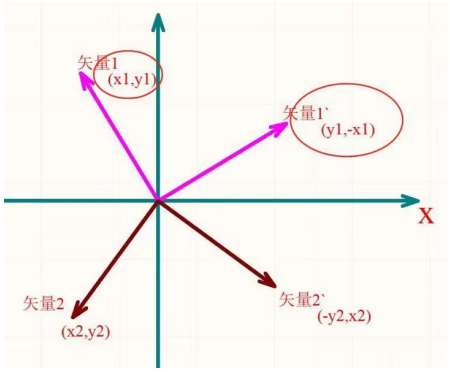
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    bra         CordicSetup
QuadIIIQuadAdjust:
    neg         w0,w1
    mov         [w9],w0
    mov         #NEG_PI_DIV_2,ACCUM_ANGLE ;第三象限变到第四象限，逆时针旋转90度，此时初始角度是-90度
CordicSetup:
```

坐标调整之后呢，初始角度就是-90°。所以，需要把这个值赋值给角度角度累加器。这样，就实现了第三象限的处理。

```
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整 第二象限调整到第一象限，第三象限调整到第四象限
    cp0         w1
    bra         NN,CordicSetup
    mov         w1,[w9]
    cp0         w0
    bra         LE,QuadIIIQuadAdjust
    mov         w0,w1
    neg         [w9],w0
    mov         #PI_DIV_2,ACCUM_ANGLE  ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra         CordicSetup
QuadIIIQuadAdjust:
    neg         w0,w1
    mov         [w9],w0
    mov         #NEG_PI_DIV_2,ACCUM_ANGLE ;第三象限变到第四象限，逆时针旋转90度，此时初始角度是-90度
CordicSetup:
```

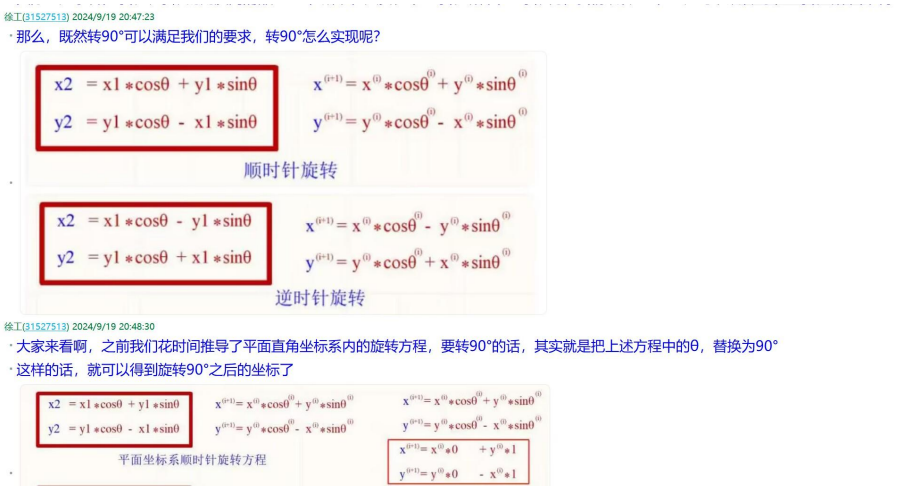
然后，如果是第二象限的话，就要顺时针旋转 90°到第一象限。



第二象限旋转到第一象限的话，就是　原始坐标的 y 变为新坐标的 x，原始坐标的 x 的相反数变为新坐标的 y。

问：旋转象限后为什么要 y 变 x，x 变 y？没有理解
答：这个在前面讲理论的时候有推导。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



你可以按照上面的时间找一下。



```
    bra     LE, QuadIIIQuadAdjust
    mov     w0, w1
    neg     [w9], w0
    mov     #PI_DIV_2, ACCUM_ANGLE   ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra     CordicSetup
QuadIIIQuadAdjust:
    neg     w0, w1
```

所以，这里就直接把 y 赋值给 w1，作为 x。



```
    mov     w0, w1
    neg     [w9], w0
    mov     #PI_DIV_2, ACCUM_ANGLE   ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra     CordicSetup
```

然后，原来的 x 取反之后，给到 w0，作为 y



```
    mov     w0, w1
    neg     [w9], w0
    mov     #PI_DIV_2, ACCUM_ANGLE   ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra     CordicSetup
QuadIIIQuadAdjust:
```

此时，对应的初始角度是 90°

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整 第二象限调整到第一象限，第三象限调整到第四象限
    cp0         w1
    bra         NN,CordicSetup
    mov         w1,[w9]
    cp0         w0
    bra         LE,QuadIIIQuadAdjust
    mov         w0,w1
    neg         [w9],w0
    mov         #PI_DIV_2,ACCUM_ANGLE  ;第二象限变到第一象限，顺时针旋转90度，此时初始角度是90度
    bra         CordicSetup
QuadIIIQuadAdjust:
    neg         w0,w1
    mov         [w9],w0
    mov         #NEG_PI_DIV_2,ACCUM_ANGLE ;第三象限变到第四象限，逆时针旋转90度，此时初始角度是-90度
CordicSetup:
```

所以，到目前为止，象限的调整就处理好了，矢量都变到第一象限或者第四象限了，下面的任务呢，就是计算迭代方程。

```
QuadIIIQuadAdjust:
    neg         w0,w1
    mov         [w9],w0
    mov         #NEG_PI_DIV_2,ACCUM_ANGLE ;第三象限变到第四象限，逆时针旋转90度，此时初始角度是-90度
CordicSetup:
    lac         w0,#1,B
    lac         w1,#1,A
    movpag      #psvpage(_Cordic_Table),DSRPAG
    mov         #psvoffset(_Cordic_Table),w8
    mov         #0x7FFF,w5
    do          #9,EndCordic
    sac.r       A,[w9]
    sac.r       B,w6
    cp0         w6
    bra         N,rotate_neg ;y<0，逆时针旋转，y>0,顺时针旋转
rotate_pos: ;顺时针旋转
```

然后，我们继续来看啊，象限调整完毕之后，就要准备计算迭代方程了。

那我们还是继续啊，课后大家再仔细看一下，不明白的地方可以问助教，也可以下节课问我。计算迭代方程前，我们要做一些准备工作。因为我们迭代要使用到累加器，所以需要把 x　y 的初值给装载到累加器中。

```
CordicSetup:
    lac         w0,#1,B
    lac         w1,#1,A
    movpag      #psvpage(_Cordic_Table),DSRPAG
    mov         #psvoffset(_Cordic_Table),w8
    mov         #0x7FFF,w5
    do          #9,EndCordic
    sac.r       A,[w9]
    sac.r       B,w6
    cp0         w6
    bra         N,rotate_neg ;y<0，逆时针旋转，y>0,顺时针旋转
rotate_pos: ;顺时针旋转
```

大家看下，这两句话就是把 x 跟 y 装载到累加器，y 装载到累加器 B，x 装载到累加器 A。只不过，这里装载的时候呢，同时把 x 跟 y 缩小了一半。因为　arctan(y/x),x,y 同时缩小一半的话，结果是不变的。缩小的话，也是为了让累加器累加更安全，不至于数据累加溢出。

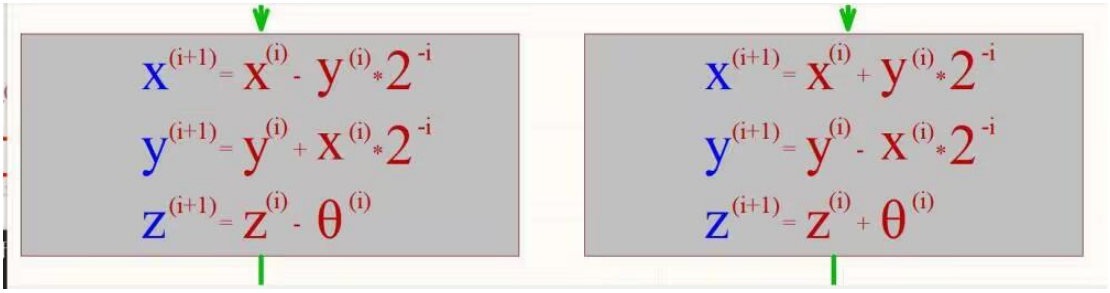
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
CordicSetup:
    lac         w0,#1,B
    lac         w1,#1,A
    movpag      #psvpage(_Cordic_Table),DSRPAG
    mov         #psvoffset(_Cordic_Table),w8
    mov         #0x7FFF,w5
    do          #9,EndCordic
    sac.r       A,[w9]
    sac.r       B,w6
    cp0         w6
    bra         N,rotate_neg ;y<0，逆时针旋转，y>0,顺时针旋转
rotate_pos: ;顺时针旋转
```

接着呢，是获取查表的首地址。因为每次旋转的角度是存在 FLASH 中的，所以，我们每次旋转的时候，需要把对应的旋转角度取出来，参与角度累加的计算。因此，就需要知道角度表的首地址。

```
CordicSetup:
    lac         w0,#1,B
    lac         w1,#1,A
    movpag      #psvpage(_Cordic_Table),DSRPAG
    mov         #psvoffset(_Cordic_Table),w8
    mov         #0x7FFF,w5
    do          #9,EndCordic
    sac.r       A,[w9]
```

接着呢，这句话是获取 i = 0 时的 2^(-i)的值。对于 Q15 来说，32767 就对应着 1，所以，这里也就把 32767 赋值给 w5，w5 也就对应迭代方程中的 2^(-i)。



所以大家看下，为了要计算这个迭代方程，对应的初始条件值现在都确定下来了。下面就可以直接计算迭代方程了。

```
    mov         #0x7FFF,w5
    do          #9,EndCordic
    sac.r       A,[w9]
    sac.r       B,w6
    cp0         w6
    bra         N,rotate_neg ;y<0，逆时针旋转，y>0,顺时针旋转
rotate_pos: ;顺时针旋转
```

那么，这里又再一次取出 x 跟 y 放在[w9]跟 w6 中。因为计算旋转方程的时候，我们需要知道是逆时针还是顺时针，此时呢，我们就需要根据 y 的正负来判断旋转方向。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    mov         #0x7FFF,w5
    do          #9,EndCordic
    sac.r       A,[w9]
    sac.r       B,w6
    cp0         w6
    bra         N,rotate_neg ;y<0，逆时针旋转，y>0,顺时针旋转
rotate_pos: ;顺时针旋转
    mac         w5*w6,A,[w9],w6
    msc         w5*w6,B,[w8]+=2,w4
```

所以，这里就判断 w6 寄存器中数值的正负。如果是负的话，说明 y 小于 0，此时呢，需要逆时针旋转，才能靠近 x 轴的正半轴。

```
    sac.r       A,[w9]
    sac.r       B,w6
    cp0         w6
    bra         N,rotate_neg ;y<0，逆时针旋转，y>0,顺时针旋转
rotate_pos: ;顺时针旋转
    mac         w5*w6,A,[w9],w6
    msc         w5*w6,B,[w8]+=2,w4
    add         w4,ACCUM_ANGLE,ACCUM_ANGLE
    bra         EndCordic
rotate_neg: ;逆时针旋转
    msc         w5*w6,A,[w9],w6
    mac         w5*w6,B,[w8]+=2,w4
    subbr       w4,ACCUM_ANGLE,ACCUM_ANGLE
EndCordic:
    lsr         w5,w5
    mov         ACCUM_ANGLE,w0
```

所以，逆时针的话就是跳转到红色箭头的部分。

$$x^{(i+1)} = x^{(i)} - y^{(i)} * 2^{-i}$$
$$y^{(i+1)} = y^{(i)} + x^{(i)} * 2^{-i}$$
$$z^{(i+1)} = z^{(i)} - \theta^{(i)}$$

此时，对应使用的迭代方程就是这个， 第一个方程，是计算一个减法。

```
    bra         EndCordic
rotate_neg: ;逆时针旋转
    msc         w5*w6,A,[w9],w6
    mac         w5*w6,B,[w8]+=2,w4
    subbr       w4,ACCUM_ANGLE,ACCUM_ANGLE
EndCordic:
```

这里用了一条 msc 指令， 它是相乘并累减。因为，初始时刻的 x 是存储在累加器 A 中的。而 y 是存储在 w6 中的，2^（-i）是存在 w5 寄存器中的，所以，w5*w6 对应的就是 2^(-i)*y。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

MSC	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd,AWB	相乘并从累加器中减去乘积	1	1	301

那么，这条 msc 指令的功能，是相乘并从累加器中减去乘积。累加器 A 存储的是 x，所以指令执行后，也就是实现 x - 2^(-i)*y。得到的结果呢，还是存储在累加器中。

语法：	{标号:} MSC Wm * Wn, Acc	{,[Wx], Wxd}	{,[Wy], Wyd}	{,AWB}
		{,[Wx] + = kx, Wxd}	{,[Wy] + = ky, Wyd}	
		{,[Wx] – = kx, Wxd}	{,[Wy] – = ky, Wyd}	
		{,[W9 + W12], Wxd}	{,[W11 + W12],Wyd}	
操作数：	Wm * Wn ∈ [W4 * W5, W4 * W6, W4 * W7, W5 * W6, W5 * W7, W6 * W7] Acc ∈ [A,B] Wx ∈ [W8, W9]； kx ∈ [-6, -4, -2, 2, 4, 6]； Wxd ∈ [W4 ... W7] Wy ∈ [W10, W11]； ky ∈ [-6, -4, -2, 2, 4, 6]； Wyd ∈ [W4 ... W7] AWB ∈ [W13, [W13] + = 2]			
操作：	(Acc(A 或 B)) – (Wm) * (Wn) → Acc(A 或 B) ([Wx]) → Wxd； (Wx) + kx → Wx ([Wy]) → Wyd； (Wy) + ky → Wy 舍入后的 (Acc(B 或 A)) → AWB			
受影响的状态位：	OA、OB、OAB、SA、SB 和 SAB			

大家看下这条指令的操作描述就清楚了。所以，第一个迭代方程，就使用一条 msc 指令来实现的。而这样复杂的计算，只需 1 个机器周期就可以实现！

今天先暂时不讲了，大家课后再慢慢消化一下，看看大家有没有什么问题？剩下的内容下节课再讲。这种汇编的代码实现 CORDIC 编程在低档单片机实现 FOC 控制比较有用。大家课后把 CORDIC 剩下的代码内容预习一下，下节课再把迭代方程讲完。

今天就到这里，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。